Distributed Systems · Monsoon 2026 · Course project, Option 1

# Chord vs Pastry

Comparing two ways to find data in a peer-to-peer network, built from the original papers

Prateek Goyal · Abhinav Bachchas · Devansh Pandey

Mid-semester presentation

# The problem: finding data with no central server

Thousands of computers each store some files.

Given a file name, which computer has it?

**No central server**

Nothing in the middle that can crash or get overloaded.

**Small memory**

Each computer knows only a few other computers.

**Few steps**

Any file is found in about log N messages.

**Keeps working**

Computers join, leave and crash all the time.

# Motivation and our hypothesis

### Amazon Dynamo, Cassandra

Store data on a hash ring, the same idea as Chord’s consistent hashing.

### BitTorrent, IPFS, Ethereum

Find peers and files with Kademlia, a later DHT built on these ideas.

### Two award-winning papers

Both won 10-year awards in 2011, yet solve the same problem in very different ways.

**Our hypothesis**

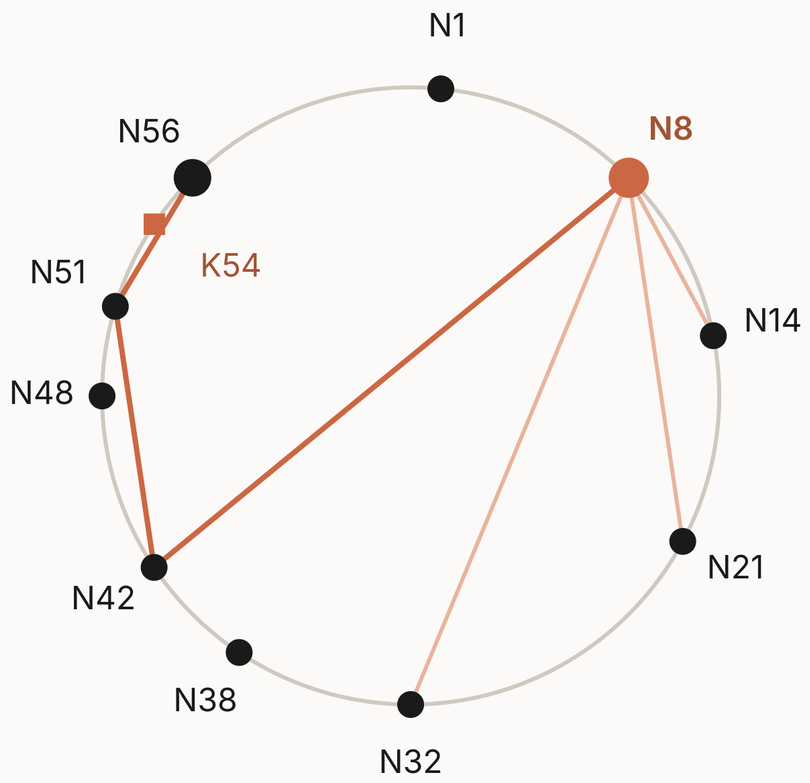
We test it by measuring hops, delay, memory per node, join cost and behaviour under failures.

Pastry is faster when network delays differ a lot, because it picks nearby nodes.

Chord is simpler and cheaper to keep correct.

# Paper 1, Chord: the architecture

Stoica et al., ACM SIGCOMM 2001, longer version in IEEE/ACM Transactions on Networking 2003




**Finger table of node N8 (ring of $2^6$ = 64 IDs)**

| Start = 8 + $2^i$ | First node after it |
|---|---|
| 9, 10, 12 | N14 |
| 16 | N21 |
| 24 | N32 |
| 40 | N42 |

Key K54 is stored at its **successor**, N56.

A lookup from N8 goes N8 → N42 → N51 → N56. Each jump is the farthest finger that does not pass the key.

Redrawn from Fig. 4 of the Chord paper. Light lines: N8's fingers. Dark orange: the lookup path.

# Chord: lookup, joining and failures

```
Lookup (paper Fig. 5, simplified)
n.find_successor(id):
  if id in (n, successor]:
    return successor
  n2 = closest_preceding_node(id)
  return n2.find_successor(id)

n.closest_preceding_node(id):
  for i = m down to 1:
    if finger[i] in (n, id):
      return finger[i]
  return n
```

**Join:** a new node asks any node to find its successor, then links in.

**Stabilize** (every second): ask the successor for its predecessor; if a new node is in between, use it. Then notify the successor.

**Fix fingers** refreshes one finger at a time. **Successor list** of r nodes handles crashes.

### What the paper proves

O(log N) hops with high probability (Thm IV.2). Joins plus stabilize always fix the ring (Thm IV.3). With r = O(log N), lookups work even if half the nodes fail (Thm IV.5).

# Paper 2, Pastry: the architecture

Rowstron and Druschel, IFIP/ACM Middleware 2001. What one node with ID d46a1c... keeps:

### Routing table: row l, column c holds a node that shares my first l digits and has c next

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| row 0: _ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | **d** | e | f |
| row 1: d_ | 0 | 1 | 2 | 3 | **4** | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
| row 2: d4_ | 0 | · | 2 | · | 4 | 5 | **6** | · | 8 | · | a | · | c | · | e | · |

Orange = my own digit (no entry needed). Dots = no such node yet. Real IDs have 32 hex digits, so up to 32 rows; only about $\log_{16}N$ rows are filled.

### Leaf set (16 nodes)

The 8 closest smaller IDs and 8 closest larger IDs, e.g. d46a02, d46a17 | d46a2c, d46a51. Gives the last hop and keeps routing correct.

### Neighbourhood set (32 nodes)

The nodes with the lowest network delay to me, whatever their ID. Used to keep the routing table local.

Based on Fig. 1 of the Pastry paper, which shows the same three parts (it uses base 4 and short IDs to keep the picture small).

# Pastry: routing, joining and locality

```
Routing at node A for key D (paper Table 1, simplified)
if D is inside the leaf set range:
  forward to the leaf closest to D
else:
  l = shared prefix length(D, A)
  if R[l][digit l of D] exists:
    forward to that node
  else: # the rare case
    forward to any known node T with
      prefix(T, D) ≥ l and
      |T - D| < |A - D|
```

Example route for key d46a1c (our own example)

65a1fc → d13da3 → d4213f → d462ba → d467c4

Each hop matches one more digit. The last hop uses the leaf set: d467c4 is the closest ID to the key.

---

**Join:** the new node sends a join message to its own ID through a nearby node. Each node on the way gives it table rows; the last one gives its leaf set.

**Locality:** for every table entry, keep the node with the lowest delay.

# Same goal, two different designs

| | Chord | Pastry |
|---|---|---|
| Who stores a key | Next node clockwise (successor) | Node with the closest ID |
| Next hop | Finger closest to the key, without passing it | Node with one more matching digit |
| What a node remembers | About $\log_2 N$ fingers + r successors | Routing table + 16 leaves + 32 nearby nodes |
| Expected hops | About $\frac{1}{2} \cdot \log_2 N$ | About $\log_{16} N$ |
| Uses physical distance | No | Yes, prefers nearby nodes |
| Repair | Stabilize and fix fingers every second | Leaf-set checks; table fixed only when used |

# Related and recent work

| Paper | What they did and found | What it means for us |
|---|---|---|
| **Gummadi et al.**<br>SIGCOMM 2003 | Compared DHT routing shapes (tree, hypercube, ring, XOR…). The ring is the most flexible: best resilience to failures and best use of nearby nodes. | Chord is a ring, so it could also choose nearby fingers. It doesn't, which explains part of the gap we see. |
| **Zave**<br>CCR 2012, IEEE TSE 2017 | Checked Chord with a formal model (Alloy). No published version is fully correct: some orders of joins and failures break the ring. Gave a fixed version with proofs. | Our Chord follows the original, so it may have the same rare bugs even though our tests pass. We will test her cases. |
| **Kanemitsu, Nakazato**<br>IFIP Networking 2021 | KadRTT: in Kademlia, pick the lowest-delay next hop that still gets close enough to the key. Lower delay and fewer hops in simulation. | Pastry's locality idea, 20 years later. Our "nearby vs random tables" test measures the same effect. |
| **Trautwein et al.**<br>SIGCOMM 2022 | Measured IPFS, which finds content with a Kademlia DHT: peers in 2,700+ networks in 152 countries, millions of retrievals a day. | DHTs work at real scale, and real nodes join and leave constantly, so our next phase adds churn. |

Other DHT designs: CAN (SIGCOMM 2001), Kademlia (IPTPS 2002), Tapestry (IEEE JSAC 2004).

# Our work: how we built and compared them

**Topology**
Each node gets an (x, y) spot on a map

→

**Network**
Delay = distance, plus loss and crashes

→

**Chord and Pastry**
Written by us from the papers

→

**Metrics**
Same counters for both overlays

## From the paper to our code

| | |
|---|---|
| Chord: find_successor, closest_preceding_node | same names, plus next_hop() |
| Chord: join, stabilize, notify, fix_fingers | same names in ChordNode |
| Pastry: routing rule (Table 1) | PastryNode.next_hop() |
| Pastry: node arrival and repair | join(), forget(), repair_leaf() |
| Both: forwarding, acks, timeouts | route() in sim/overlay.py |

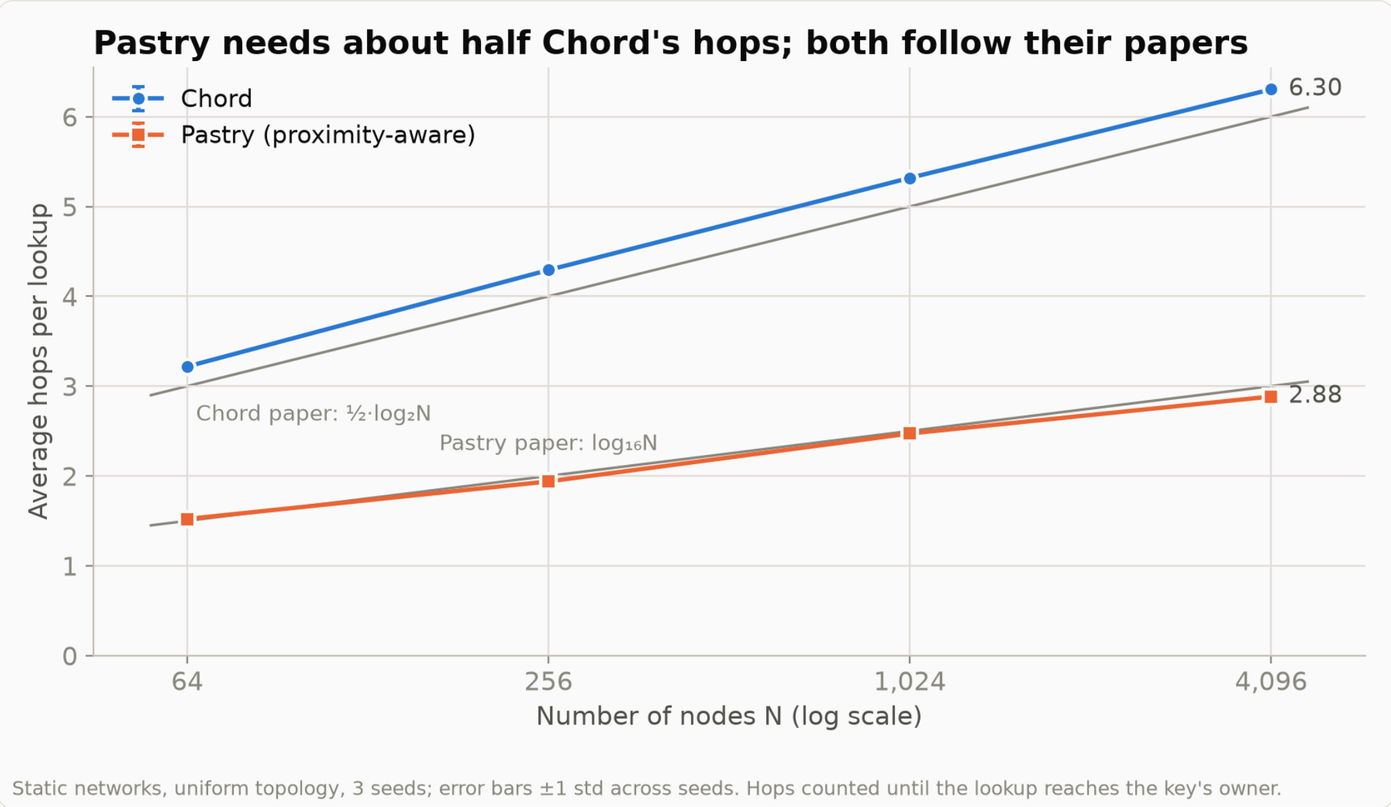
## Fair, tested, honest

Same routing code, counters and node positions for both. Three maps, one with equal delays as a control.

116 tests; a checker confirms every lookup: 100% correct.

Python, simpy, SHA-1, pytest, matplotlib. No DHT library.

# Result 1: hops grow the way the papers say




Static networks, uniform topology, 3 seeds; error bars ±1 std across seeds. Hops counted until the lookup reaches the key's owner.

Chord, 4,096 nodes

**6.30**

hops (paper: ½·$\log_2 N$ = 6.0)

Pastry, 4,096 nodes

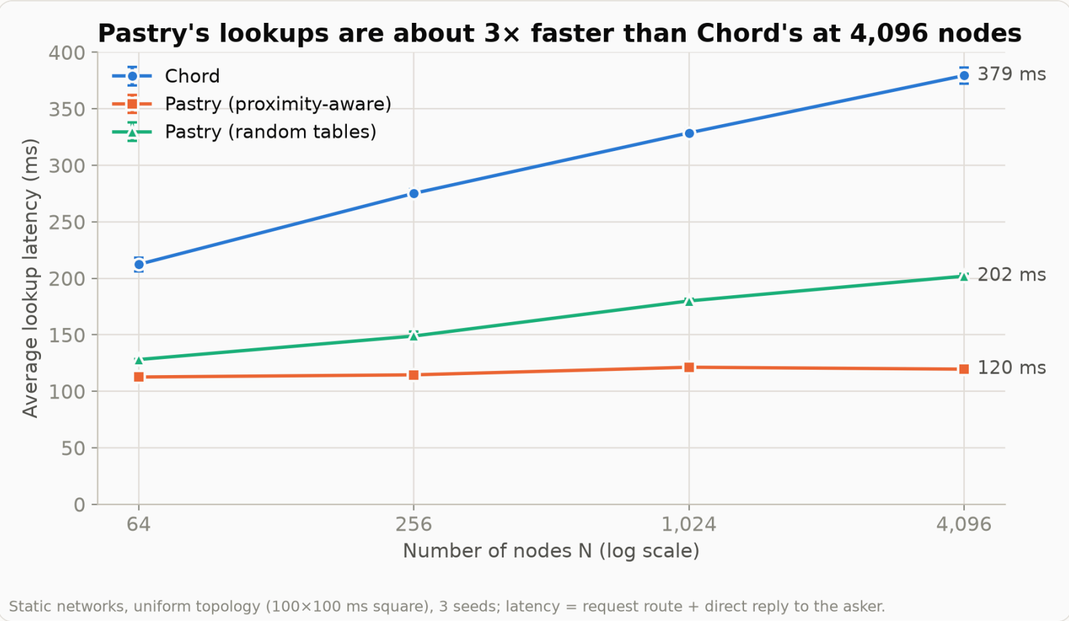
**2.88**

hops (paper: $\log_{16} N$ = 3.0)

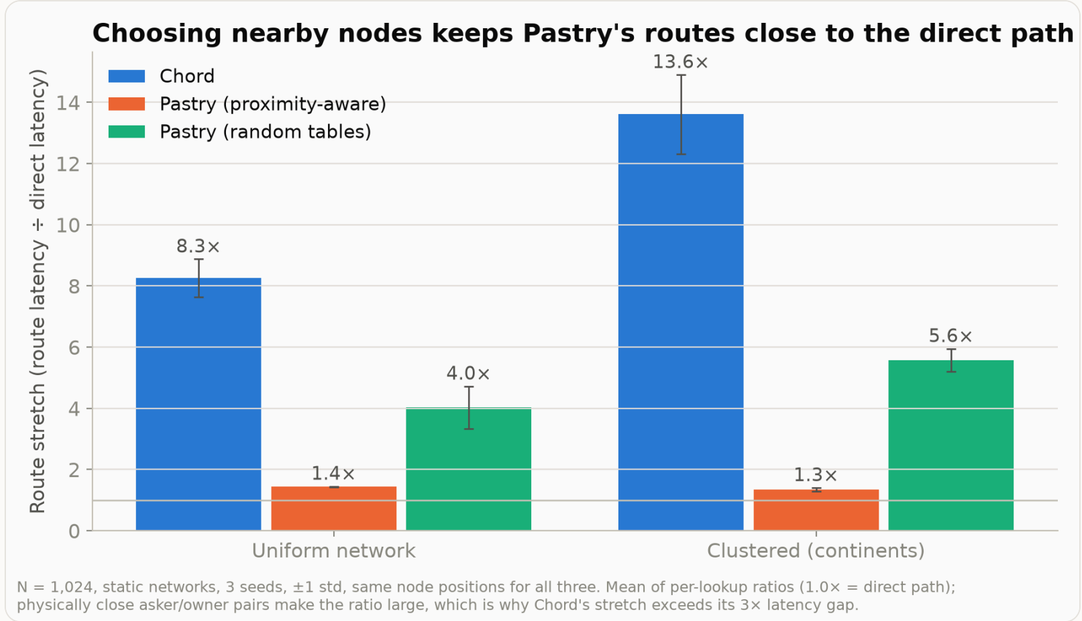
The slopes match both papers. With the Chord paper's own setting (1,000 nodes, 20 successors) we get 3.72 hops; the paper reports 3.84.

Stable networks, 3 seeds, every lookup checked and correct.

# Result 2: about 3× faster, thanks to nearby nodes



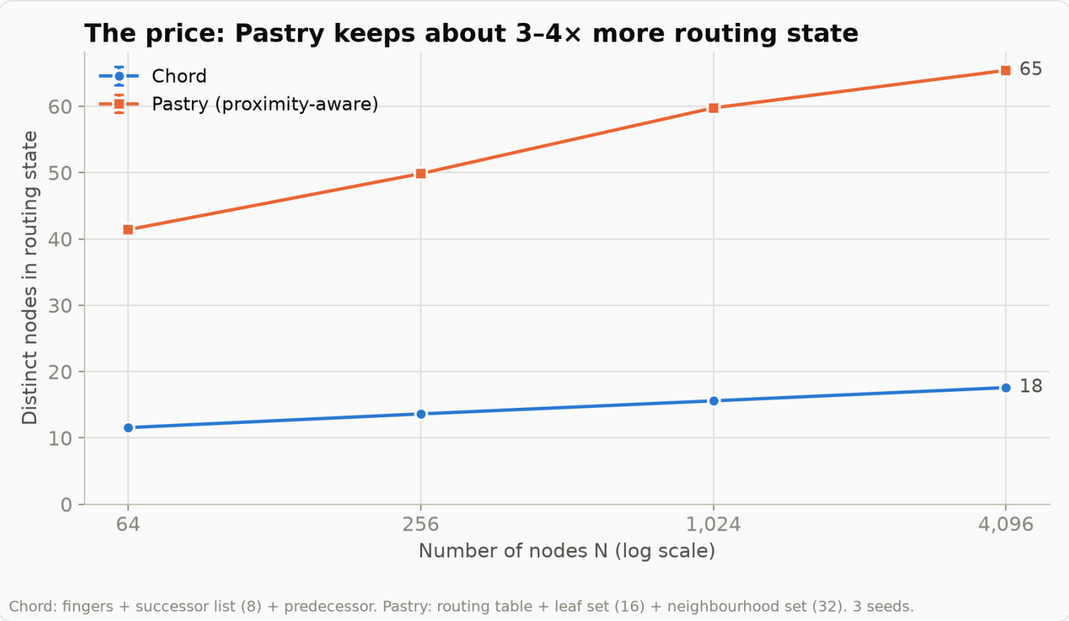

**Delay at 4,096 nodes:** Chord 379 ms, Pastry 120 ms. Pastry with random tables: 202 ms.



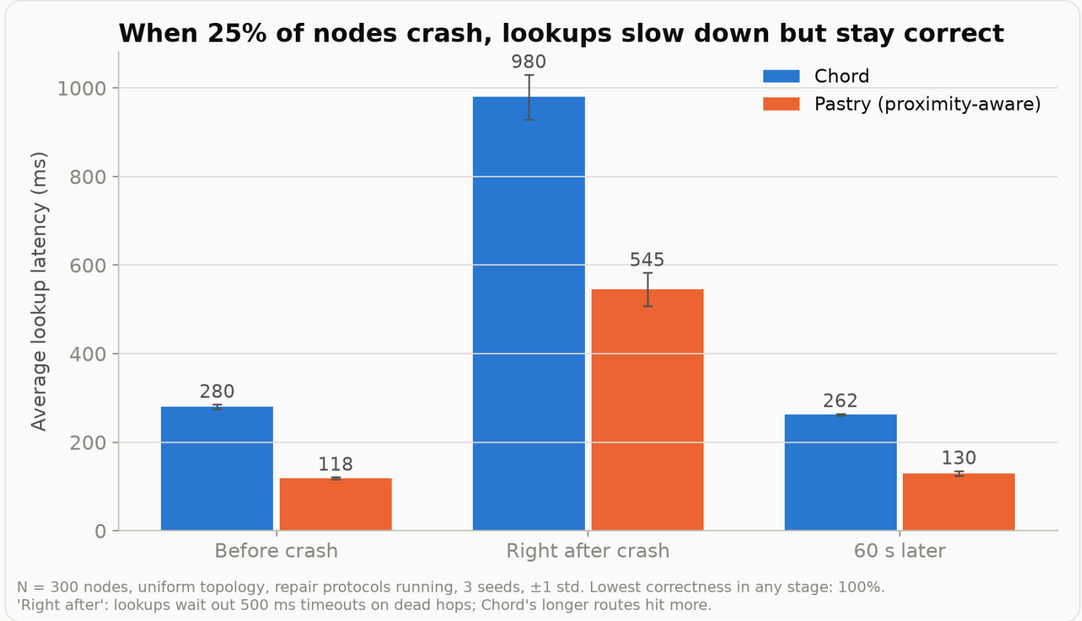

**Route stretch:** Pastry 1.4× the direct delay; 4.0× with random tables, same hops.

Left: evenly spread map. Right: 1,024 nodes, stretch = route delay ÷ direct delay. Fewer hops explain part of the gap, nearby nodes the rest.

# Result 3: the price of Pastry, and crashes




**More memory:** Pastry keeps 65 nodes in its tables, Chord only 18.




**25% of nodes crash:** both stay 100% correct. Chord slows down more, then both recover.

Crash test: 300 nodes, repair running, 3 seeds.

# Summary, limits and next steps

Hops at 4,096 nodes

6.30 → 2.88

Lookup delay

379 → 120 ms

Nodes remembered

18 → 65

Each pair is Chord → Pastry. Our hypothesis holds so far: Pastry is faster, Chord keeps less state and repairs faster.

### Honest limits

No churn yet. Pastry reads delay for free. Only 3 seeds. Zave’s cases not tested.

### Next

Key-value store and churn, full experiments with more seeds, then the IEEE report and the final demo.

# References


[1] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, H. Balakrishnan. Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications. ACM SIGCOMM 2001; IEEE/ACM Trans. on Networking, 2003.

[2] A. Rowstron, P. Druschel. Pastry: Scalable, Decentralized Object Location and Routing for Large-Scale Peer-to-Peer Systems. IFIP/ACM Middleware 2001.

[3] S. Ratnasamy et al. A Scalable Content-Addressable Network. ACM SIGCOMM 2001.

[4] P. Maymounkov, D. Mazières. Kademlia: A Peer-to-peer Information System Based on the XOR Metric. IPTPS 2002.

[5] K. Gummadi et al. The Impact of DHT Routing Geometry on Resilience and Proximity. ACM SIGCOMM 2003.

[6] P. Zave. Using Lightweight Modeling to Understand Chord. ACM SIGCOMM Computer Communication Review, 2012.

[7] P. Zave. Reasoning about Identifier Spaces: How to Make Chord Correct. IEEE Trans. on Software Engineering, 2017.

[8] H. Kanemitsu, H. Nakazato. KadRTT: Routing with Network Proximity and Uniform ID Arrangement in Kademlia. IFIP Networking 2021.

[9] D. Trautwein et al. Design and Evaluation of IPFS: A Storage Layer for the Decentralized Web. ACM SIGCOMM 2022.


**Thank you. Questions are welcome.**